

Vectorless RAG Documentation

Vectorless RAG — Concept, Implementation, and Test Results

1. What Problem This Solves

Standard RAG converts document chunks and user queries into vectors (embeddings) and retrieves by mathematical similarity — comparing how close two pieces of text are in vector space. This works, but it has a known failure mode: **similarity is not the same as relevance**.

A concrete example: in a contract, the clause that *creates* a warranty and the clause that *voids* a warranty can both contain the words “warranty,” “void,” and “conditions.” A vector search can retrieve the wrong one, because both score as mathematically similar to a query about warranties — even though only one is logically correct.

Vectorless RAG (the approach used by tools like PageIndex) avoids this by never comparing meaning-as-numbers. Instead, it builds a table of contents — a tree of section titles — and has an LLM *reason* about which section is relevant, the way a person flips to a chapter title instead of scanning every page. No embeddings, no vector database, no chunk-size tuning.

Important distinction: this project does **not** use the PageIndex library or API. It is a from-scratch implementation of the same underlying idea, built to understand the mechanism directly. See Section 7 for how this differs from the real PageIndex tool.

2. The Trade-off

	Vector RAG	Vectorless RAG
Setup cost	Embedding model + vector DB + chunking strategy	None — just parse structure
Per-query cost	1 similarity lookup	Multiple LLM calls (2-3 here)
Transparency	A similarity score, no explanation	A visible reasoning trail — you see exactly which chapter/section it walked through and why
Depends on	Nothing about document structure	The document actually HAVING real structure (headings/hierarchy)

If a document has no real structure — scraped web pages, chat logs, unstructured notes — there’s no table of contents to reason over, and vector search is the better tool. This approach specifically targets long, *structured* documents: manuals, contracts, filings.

3. Pipeline Overview

disk file (nested_manual.md)

```

| open() + read()
v
one big string
| split by lines
v
list of lines
| classify: "# " = chapter, "## " = subsection, else = body text
v
nested tree: chapters -> subsections -> text      (load_tree() – 0 LLM calls)
| LLM call #1: pick a chapter from ~8 titles
v
one chapter (with its subsections)                (pick() – hop 1)
| LLM call #2: pick a subsection from that chapter's titles
v
one subsection (with its full text)                (pick() – hop 2)
| LLM call #3: answer using ONLY that subsection's text
v
final answer                                      (answer())

```

Total cost per question: **3 LLM calls, 0 embeddings, 0 vector lookups.**

4. Tech Stack

- **LLM provider:** Groq API (free tier, no credit card required)
- **Model:** openai/gpt-oss-20b (Note: llama-3.3-70b-versatile and llama-3.1-8b-instant are deprecated on Groq as of this writing — confirmed by hitting a live 404 during setup.)
- **Orchestration:** LangChain (langchain-groq , ChatGroq , HumanMessage)
- **Test document:** nested_manual.md — a hand-written markdown file, 8 chapters, each with 3–4 subsections (29 subsections total)

5. Code Walkthrough

5.1 Parsing the document into a tree (load_tree)

```

def load_tree(path: str):
    with open(path, "r", encoding="utf-8") as f:
        raw = f.read()

    chapters = []
    current_chapter = None
    current_sub = None

    for line in raw.split("\n"):
        if line.startswith("# "):
            current_chapter = {"title": line[2:].strip(), "subsections": []}
            chapters.append(current_chapter)
            current_sub = None

        elif line.startswith("## "):
            current_sub = {"title": line[3:].strip(), "text": ""}
            current_chapter["subsections"].append(current_sub)

        else:
            if current_sub is not None:
                current_sub["text"] += line + "\n"

    for chapter in chapters:
        for sub in chapter["subsections"]:
            sub["text"] = sub["text"].strip()

    return chapters

```

This makes **zero LLM calls**. It walks the file one line at a time, using two pointers (current_chapter ,

`current_sub`) to track “where am I in the structure right now.” A subsection is nested inside a chapter simply because it’s appended into that chapter’s own `subsections` list — that nesting *is* the tree. This is the vectorless replacement for “build embeddings and populate a vector database”: it runs in milliseconds regardless of document size, because it’s pure string parsing.

Verified: parsing `nested_manual.md` correctly produces 8 chapters with the correct subsection counts and text bodies for each (manually checked against the source file).

5.2 Title-only indexes (`chapter_index` , `subsection_index`)

```
def chapter_index(chapters):
    return "\n".join(f"{i}. {c['title']}" for i, c in enumerate(chapters))

def subsection_index(chapter):
    return "\n".join(f"{i}. {s['title']}" for i, s in enumerate(chapter["subsections"]))
```

These expose **only titles**, never body text, to the navigation step. This keeps each navigation call cheap — the LLM reasons about structure first, and only reads actual content once, at the very last step.

5.3 The navigator (`pick`) — reused for both tree levels

```
PICK_PROMPT = """You are navigating a table of contents.
Given the user's question, pick the ONE entry most likely to contain the answer.

Entries:
{index}

Question: {question}

Reply with ONLY a JSON object: {"choice": <int>}}
"""

def pick(llm, index: str, question: str) -> int:
    prompt = PICK_PROMPT.format(index=index, question=question)
    response = llm.invoke([HumanMessage(content=prompt)])
    raw = re.sub(r"^```json|```$", "", response.content.strip(), flags=re.MULTILINE).strip()
    try:
        return int(json.loads(raw)["choice"])
    except (json.JSONDecodeError, KeyError, ValueError):
        raise RuntimeError(f"Navigator returned something unparseable: {raw!r}")
```

This is the same function called twice: once against the 8 chapter titles (hop 1), once against the chosen chapter’s subsection titles (hop 2). The strict JSON output format matters — without it, models tend to add explanation text that breaks automated parsing.

5.4 Final answer generation (`answer`)

```
ANSWER_PROMPT = """Answer the question using ONLY the section below.
If the section does not contain the answer, say so -- do not guess.

Section: {title}
---
{text}
---

Question: {question}
"""

def answer(llm, subsection: dict, question: str) -> str:
    prompt = ANSWER_PROMPT.format(title=subsection["title"], text=subsection["text"], question=question)
    response = llm.invoke([HumanMessage(content=prompt)])
    return response.content.strip()
```

By this point, the entire document has been narrowed down to one subsection. Only that subsection’s text reaches this prompt — nothing else from the document is ever shown to the model here. The explicit “don’t guess” instruction is what keeps this from hallucinating when the navigator picks the wrong branch (see Test 5 below).

6. Test Results (live runs against Groq, `openai/gpt-oss-20b`)

Five questions were run end-to-end. Four succeeded correctly; one revealed a real, explainable limitation.

#	Question	Hop 1 (chapter)	Hop 2 (subsection)	Result
1	How long are user sessions valid for?	2. Authentication ✓	2.4 Session Expiry and Revocation ✓	Correct — “24 hours of inactivity”
2	What happens if I exceed my API rate limit?	7. Rate Limiting and Quotas ✓	7.1 Request Rate Limits ✓	Correct — HTTP 429, Retry-After header
3	Why does the system use a secondary ranking step after similarity search?	5. Retrieval Pipeline ✓	5.4 Re-Ranking ✓	Correct — and notably avoided the near-miss of 5.3 Query-Time Retrieval, which also mentions similarity search but isn’t the re-ranking step being asked about
4	What programming language is the backend written in? (not in the document)	1. System Architecture	1.2 Deployment Topology	Correctly answered “the section doesn’t specify” instead of guessing
5	What’s the process if someone’s session needs to be terminated urgently after a security issue?	8. Monitoring and Incident Response ✗	8.3 Incident Severity Levels	Wrong chapter. Correct answer lives in 2.4 (session revocation). Navigator matched question <i>vocabulary</i> (“urgently,” “security issue” → “incident”) rather than the actual <i>mechanism</i> (session revocation). Given the wrong branch, hop 2 and the answer step behaved correctly within it — the system said “I don’t have that information” rather than fabricating an answer from the wrong section.

Test 5 is the most useful result in this whole exercise. It's a real, reproducible failure with a clear explanation, and it demonstrates something important: the system **fails safely** (wrong retrieval → honest "not found") rather than **failing silently** (wrong retrieval → confident hallucination). That distinction matters more than a clean 5/5 record would.

7. Known Limitations

1. **No backtracking.** If hop 1 picks the wrong chapter, the system commits to it — there's no mechanism to detect "hop 2 didn't find a convincing answer, try a different chapter." Test 5 above is a live example of this. A production system would likely retry against a second candidate branch.
2. **Structure-dependent.** This only works because `nested_manual.md` has clean, consistent `# / ##` markdown headings. A document without real hierarchy (scraped text, chat logs, unstructured notes) gives this approach nothing to navigate — vector search would do better there.
3. **Format-dependent, and this is a real simplification.** Markdown headings are plain-text markers (`#`) that a simple string check can detect. Real documents this technique targets — contracts, filings — usually arrive as PDF or DOCX, where "this is a heading" is a formatting attribute (a paragraph style, font size/boldness) rather than a text pattern. Extracting structure from those formats is a genuinely harder problem this implementation does not attempt to solve.
4. **Two levels only.** This demonstrates chapter → subsection navigation. Real PageIndex-style systems often handle arbitrarily deep, irregular nesting. This script's two fixed levels are a teaching-scale simplification, not evidence the approach scales to messier real-world document structures.
5. **Rate limits.** Groq's free tier caps at 30 requests/minute and 1,000 requests/day. Fine for this project's scale (3 calls per question), but not suitable for high-volume or automated batch testing without hitting the cap.

8. Vectorless RAG vs. PageIndex (the actual library)

This project is **not** built on the PageIndex library or API — no `pip install pageindex`, no calls to `pageindex.ai`. It's an independent implementation of the same underlying idea (tree-based reasoning instead of vector similarity), for learning purposes.

Key differences from the real tool: - PageIndex can generate a structural tree even for documents **without** clean headings, using an LLM to infer reasonable structure. This implementation only works when `# / ##` markers already exist in the text. - PageIndex offers a paid cloud API (from \$30/month after a 200-credit free trial) for hosted tree generation and retrieval, but it also ships a free, local open-source mode (`pip install -U pageindex`) that runs entirely on your own machine using your own LLM key — no cloud, no payment required. This project's approach is conceptually closest to that local mode.

9. Vectorless RAG vs. Neo4j GraphRAG (for context — see companion doc)

Different axis entirely — GraphRAG models *relationships between entities* (multi-hop reasoning across a knowledge graph), while vectorless RAG models *structure within a single document* (navigating a table of contents). Neither replaces the other; they solve differently shaped problems. See the separate GraphRAG documentation for that implementation and its own test results.